AN INTEGRATED EXPLAINABLE AI PLATFORM FOR ACADEMIC GUIDANCE, CAREERS, AND WELLBEING

INDIVIDUAL THESIS

Wickremathilaka J A
(IT22152114)

Individual component: Career Pathway and Skill Gap Analyzer
Group ID: R26-IT-073

Report submitted in partial fulfilment of the requirements for the
Bachelor of Science (Hons) in Information Technology
Specializing in Information Technology

Department of Information Technology
Sri Lanka Institute of Information Technology
Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy (Appendix B). I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Wickremathilaka J A
Student ID: IT22152114

Signature:  Date: 10/10/2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Ms. Thilini Jayalath		
Co-supervisor: Ms. Jenny Krishara		

Abstract

Students entering the IT labor market must translate their skills and degree into realistic career choices, yet existing tools rarely quantify the gap to a target role or explain their advice. This research develops the Career Pathway and Skill Gap Analyzer, the career guidance component of an integrated explainable AI platform for academic guidance, careers, and wellbeing. A design-science approach was followed. Job postings for more than forty IT roles were collected from Sri Lankan and multi-portal sources, cleaned with a spaCy-based pipeline and a dictionary of over 150 skills, and enriched with curriculum-aligned and market-aligned synthetic records. A TF-IDF weighted stacking ensemble (Random Forest, linear Support Vector Machine, Multinomial Naive Bayes, and a Logistic Regression meta-learner) predicts the most suitable careers. A four-stage academic affinity cascade and a hierarchical qualification-tier resolver incorporate the student's degree and correct the "Data Science Degree Trap". A Neo4j knowledge graph supplies weighted role–skill–degree evidence from which a Skill Gap Index, market readiness, a prioritized roadmap, and a career pivot web are computed. The component is served by a FastAPI microservice and a React interface integrated into the platform.

On a stratified 20% hold-out set the stacking ensemble achieved 88.62% top-1, 96.18% top-3, and 98.71% top-5 accuracy, with macro and weighted F1-scores of 0.88 and 0.89, outperforming Random Forest (82.1%), linear SVC (84.3%), and Naive Bayes (68.4%). Every figure shown to learners is computed by documented formulas, which makes the guidance transparent. Individual contribution is an explainable, locally grounded method that combines ensemble prediction, academic affinity, and graph-based gap quantification.

Keywords: career recommendation, skill gap analysis, stacking ensemble, knowledge graph, explainable AI

ACKNOWLEDGEMENTS

I express my sincere gratitude to my supervisor, Ms. Thilini Jayalath and co-supervisor, Ms. Jenny Krishara, for their guidance, advice, and constructive feedback throughout this research and development work. Their support was essential in shaping the direction of the Career Pathway and Skill Gap Analyzer.

I thank my group members for their collaboration in integrating the individual components into a single platform, and the students who volunteered their time to take part in the evaluation of the component. I am also grateful to the academic and administrative staff of the Department of Information Technology, Sri Lanka Institute of Information Technology, for the facilities and support provided.

TABLE OF CONTENTS

Declaration and Supervisor Certification i
Abstract ii
Acknowledgements iii
Table of Contents iv
List of Figures vi
List of Tables vii
List of Abbreviations viii
1 INTRODUCTION 1
1.1 Overall Project Context 1
1.2 Individual Component Context 2
1.3 Problem Statement 2
1.4 Research Gap 3
1.5 Aim and Objectives 4
1.6 Research Questions or Hypotheses 5
1.7 Scope, Limitations, and Assumptions 5
1.8 Individual Contribution 6
1.9 Thesis Structure 7
2 LITERATURE REVIEW AND RELATED WORK 8
2.1 Review Method and Scope 8
2.2 Domain and Theoretical Foundations 8
2.3 Related Research 10
2.4 Existing Systems, Products, and Techniques 11
2.5 Gap Synthesis and Conceptual Framework 12
3 RESEARCH METHODOLOGY 13
3.1 Research Design 13
3.2 Data, Participants, and Study Materials 13
3.3 Experimental and Study Procedure 14
3.4 Measures and Evaluation Metrics 16
3.5 Data Analysis 16
3.6 Ethics, Privacy, Security, and Responsible AI 17
3.7 Reproducibility and Research Artefacts 17
4 PRODUCT AND SOLUTION DESIGN 19
4.1 Requirements 19
4.2 Component Architecture within the Complete Solution 20
4.3 Design Alternatives and Decisions 21
4.4 Detailed Design 22
4.5 Standards and Professional Practice 28
5 IMPLEMENTATION, INTEGRATION, AND TESTING 29
5.1 Development Environment and Technology Stack 29

5.2 Component Implementation....29
5.3 Integration with the Other Components....31
5.4 Verification and Software Testing....32
5.5 Deployment and Operation....33
6 RESULTS AND VALIDATION....35
6.1 Research Results....35
6.2 Product and Technical Results....37
6.3 User or Stakeholder Validation....38
6.4 Integration Results....38
6.5 Summary of Findings....39
7 DISCUSSION....40
7.1 Interpretation of Findings....40
7.2 Comparison with Literature Baselines and Existing Solutions....40
7.3 Research Contribution....41
7.4 Product Contribution....41
7.5 Limitations and Threats to Validity....42
7.6 Implications....42
8 COMMERCIALISATION AND SUSTAINABILITY....43
8.1 Value Proposition and Target Users....43
8.2 Market and Competitive Position....43
8.3 Business and Sustainability Model....43
8.4 Scalability, Intellectual Property, and Compliance....44
8.5 Contribution to the Sustainable Development Goals....45
9 CONCLUSION AND FUTURE DIRECTIONS....46
9.1 Conclusion....46
9.2 Achievement of Objectives....46
9.3 Individual Research and Product Contributions....46
9.4 Future Directions....47
REFERENCES....48
GLOSSARY....51
APPENDICES....52
Appendix A Research Instruments....52
Appendix B Academic Integrity and AI Use Disclosure....54

LIST OF FIGURES

Figure 1.1 Position of the Career Pathway and Skill Gap Analyzer within the StudyFyX platform 1
Figure 2.1 Conceptual framework of the dual-engine career pathway analysis 12
Figure 3.1 Research and development cycle of the component 13
Figure 4.1 Layered architecture of the Career Pathway and Skill Gap Analyzer 21
Figure 4.2 Schema of the Neo4j knowledge graph 23
Figure 4.3 Four-stage academic affinity cascade 24
Figure 4.4 Structure of the Skill Synergy and Career Pivot Web (illustrative) 27
Figure 5.1 Deployment view of the component 33
Figure 6.1 Top-k accuracy of the stacking ensemble 35
Figure 6.2 Top-1 accuracy of the single baselines compared with the stacking ensemble 36

LIST OF TABLES

Table 1.1 Summary of research gaps and the corresponding design response 4
Table 2.1 Comparison of existing solution categories with the proposed component 11
Table 3.1 Data artefacts used by the component 14
Table 3.2 Model configuration 15
Table 3.3 Mapping of research questions to evaluation measures 16
Table 4.1 Functional requirements 19
Table 4.2 Non-functional requirements 20
Table 4.3 Design alternatives and decisions 22
Table 4.4 Qualification tier hierarchy 25
Table 4.5 Worked example of readiness and SGI under three-degree outcomes (illustrative) 26
Table 4.6 REST endpoints of the career AI service 27
Table 4.7 Standards and good practice applied 28
Table 5.1 Technology stack 29
Table 5.2 Integration points with the platform 32
Table 5.3 Test levels 32
Table 5.4 Principal test cases 32
Table 6.1 Performance of the stacking ensemble on the hold-out set 35
Table 6.2 Top-1 accuracy of the baselines and the stacking ensemble 36
Table 6.3 Sample analysis output for a Computer Science student with data-oriented skills 37
Table 6.4 Usability results 38
Table 6.5 Perceived usefulness and clarity of explanation (5-point Likert scale) 38
Table 6.6 Integration checks 39
Table 6.7 Summary of findings against the research questions 39
Table 7.1 Threats to validity and mitigation 42
Table 8.1 Target user segments and their needs 43
Table 8.2 Contribution to the Sustainable Development Goals 45
Table 9.1 Achievement of the objectives 46
Table B.1 AI tool use 54

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
BI	Business Intelligence
CORS	Cross-Origin Resource Sharing
CS	Computer Science
CSV	Comma-Separated Values
ESCO	European Skills, Competences, Qualifications and Occupations
ETL	Extract, Transform, Load
F1	Harmonic mean of precision and recall
FR	Functional Requirement
IDF	Inverse Document Frequency
IT	Information Technology
JSON	JavaScript Object Notation
LIME	Local Interpretable Model-agnostic Explanations
ML	Machine Learning
NB	Naive Bayes
NFR	Non-Functional Requirement
NLP	Natural Language Processing
OWASP	Open Worldwide Application Security Project
PDPA	Personal Data Protection Act
RF	Random Forest
REST	Representational State Transfer
RPA	Robotic Process Automation
RQ	Research Question
SDG	Sustainable Development Goal
SGI	Skill Gap Index
SHAP	SHapley Additive exPlanations
SLIIT	Sri Lanka Institute of Information Technology

Abbreviation	Meaning
SPA	Single-Page Application
SUS	System Usability Scale
SVC / SVM	Support Vector Classifier / Machine
TF-IDF	Term Frequency–Inverse Document Frequency
TLS	Transport Layer Security
TMW / UAW	Total Market Weight / User Achieved Weight
UI / UX	User Interface / User Experience
XAI	Explainable Artificial Intelligence

1 INTRODUCTION

1.1 Overall Project Context

Choosing a career is one of the most consequential decisions a university student makes, yet it is frequently made with incomplete information. Students in information technology (IT) programs graduate into a labor market that changes faster than most curricula, where employers advertise tens of overlapping job titles and list long, loosely structured skill requirements. At the same time, students must manage academic decisions, mental wellbeing, and career preparation simultaneously. These three concerns are usually served by separate tools, none of which explains *why* it recommends what it recommends.

The research project *An Integrated Explainable AI Platform for Academic Guidance, Careers, and Wellbeing* (Group R26-IT-073, hereafter the StudyFyX platform) addresses this fragmentation. The platform integrates three student-facing components inside a single web application: an academic guidance component, a career guidance component, and a wellbeing component. A common design principle links them: every recommendation produced by the platform must be accompanied by evidence that the student can inspect, question, and act upon. Figure 1.1 positions the individual component reported in this thesis within the overall platform.

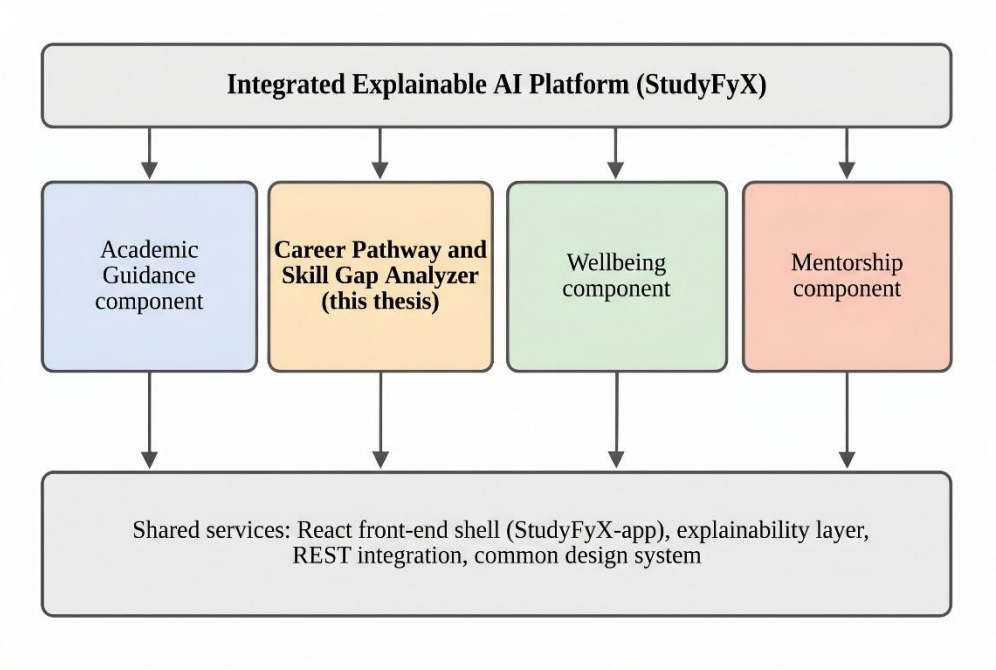


Figure 1.1: Position of the Career Pathway and Skill Gap Analyzer within the StudyFyX platform

The platform is developed by a group of four undergraduate researchers. Each member is responsible for one component and for its integration into the shared front-end shell (`StudyFyX-app`). This thesis reports the individual research and development work of the author on the **Career Pathway and Skill Gap Analyzer**, referred to in the project repository as *Member 2 – Career AI*.

1.2 Individual Component Context

The Career Pathway and Skill Gap Analyzer converts two inputs, a self-reported set of technical skills and the student's current degree, into three outputs: (i) a ranked list of the most suitable IT careers, (ii) a quantified readiness score and Skill Gap Index (SGI) for each recommended career, and (iii) a prioritized, explainable learning roadmap together with an interactive map of alternative careers that become reachable as missing skills are acquired.

The component is organized as a dual-engine pipeline with three logical stages, informally called *brains* in the project documentation:

- **Brain 1 – Destination Classifier.** A TF-IDF weighted stacking ensemble (Random Forest, linear Support Vector Machine, Multinomial Naive Bayes,

and a Logistic Regression meta-learner) that predicts the careers whose historical job postings most resemble the student's skill profile.

- **Brain 1.5 – Academic Affinity Diagnostics.** A four-stage cascade that calibrates the classifier's confidence using the student's degree, correcting cases in which a purely skill-based model ignores academic background (the "Data Science Degree Trap" described in Section 1.3).
- **Brain 2 – Pathway and Pivot Synthesizer.** A Neo4j knowledge graph and a mathematical SGI engine that compute percentage market readiness, rank missing requirements by market weight, and expose cross-career pivot opportunities (the "Synergy Web").

The analytical services are exposed through a FastAPI microservice, and the learner interacts with them through a React application consisting of an analyzer, a roadmap planner, a graph explorer, and a dashboard.

1.3 Problem Statement

Final-year and recently graduated IT students commonly face four related difficulties when planning their careers.

1. **Information overload and title ambiguity.** Job portals list dozens of near-synonymous titles (for example *Software Engineer*, *Full Stack Developer*, *Backend Developer*), and requirements are written as unstructured prose. A student cannot easily tell which role family their current skills actually support.
2. **Lack of quantified, personalized gap analysis.** Generic "roadmap" resources list everything a role might need, without distinguishing what the individual student already has, what is most heavily demanded by employers, and what to learn first.
3. **Neglect of academic background.** Skill-only recommenders treat a student with a Data Science degree and basic HTML knowledge the same as a self-taught web developer and may recommend a generic web role. Conversely, many roles carry degree expectations that a student can satisfy or exceed, which should count as evidence of readiness.

4. **Opaque recommendations.** Where existing tools do recommend careers, they rarely reveal the reasoning, the weights applied, or the evidence base. Students cannot judge whether the advice is trustworthy or how to change the outcome.

The problem addressed by this research is therefore: *how can a student's skills and degree be mapped to realistic IT careers, and how can the distance to each career be measured and explained, using evidence drawn from real job-market data and in a form that remains transparent to the student?*

1.3.1 The Data Science Degree Trap

A recurring failure mode of skill-based classifiers motivated the academic affinity stage. Basic skills such as HTML, CSS, or SQL are shared by many roles and are very frequent in job postings. A student holding a Data Science degree who lists a few of these skills can receive a high confidence "Web Developer" prediction, because the classifier sees only skills and the most frequent skills dominate. The student's degree, which is strong evidence of fit for data-oriented roles, is ignored. This thesis refers to the phenomenon as the *Data Science Degree Trap* and treats its correction as a first-class design requirement.

1.4 Research Gap

Chapter 2 reviews the literature in detail; the gaps that motivate this work can be summarized as follows. First, career recommenders in the literature are usually evaluated on top 1 accuracy over a single model, with limited evidence that ensemble methods improve robustness for sparse, high-dimensional skill vectors. Second, most systems stop at recommending a role and do not quantify the gap between the learner and the role in a form that is reproducible and auditable. Third, taxonomies such as O*NET [1] and ESCO [2] describe occupations in the United States and Europe respectively and are not grounded in the Sri Lankan job market, where employers' skill vocabularies, degree expectations, and technology stacks differ. Fourth, few systems combine a statistical classifier with a graph-structured knowledge base so that the same data both drives the prediction and supports the explanation. Finally, academic background is rarely modelled as a calibrated, explainable signal.

Table 1.1: Summary of research gaps and the corresponding design response

Gap	Observation in the literature	Design response in this component
G1: Single-model recommenders	Single classifiers are sensitive to sparse skill vectors	Stacking ensemble with out-of-fold meta-learning (Brain 1)
G2: No quantified gap	Roles are recommended without a readiness measure	Weighted SGI and market readiness (Brain 2)
G3: Local market absent	Taxonomies are US/EU centered	Corpus of job postings enriched with Sri Lankan market data
G4: Prediction and explanation separated	Explanations are produced post hoc from a black box	Shared graph supplies weights, gaps and pivots
G5: Academic background ignored	Degree rarely modelled	Affinity cascade and hierarchical qualification tiers

1.5 Aim and Objectives

1.5.1 Aim

The aim of this research is to design, implement, and evaluate an explainable career pathway and skill gap analysis component that recommends IT careers from a learner's skills and degree, quantifies the learner's readiness for each recommended career using real job-market evidence, and presents the result as an actionable roadmap.

1.5.2 Objectives

1. To construct a cleaned, multi-role job-posting corpus covering more than forty IT roles, supported by a skill-extraction pipeline with a dictionary of more than 150 categorized IT skills.
2. To develop and evaluate a TF-IDF weighted stacking ensemble that predicts the top career matches from a skill profile, and to compare it with its single-model baselines using top-1, top-3, and top-5 accuracy and macro and weighted F1-score.
3. To design a four-stage academic affinity calibration and a hierarchical qualification-tier resolver so that a learner's degree improves, rather than is ignored by, the recommendation.
4. To model roles, skills, and degrees as a Neo4j knowledge graph and to define the Skill Gap Index and Market Readiness measures computed from that graph.

5. To build a pivot analysis (the Synergy Web) that shows which alternative careers are unlocked by each missing skill.
6. To implement the analysis as a FastAPI microservice and a React front end, integrate it with the StudyFyX platform, and verify it through software testing and user validation.

1.6 Research Questions or Hypotheses

1.6.1 Research questions

- **RQ1.** Does a stacking ensemble over TF-IDF weighted skill vectors predict IT careers more accurately than each of its constituent models used alone?
- **RQ2.** Does hierarchical academic affinity calibration reduce degree-skill mismatches in the ranked recommendations?
- **RQ3.** Can a weight-based Skill Gap Index derived from a role–skill–degree graph provides gap information that learners find understandable and actionable?

1.6.2 Hypotheses

- **H1.** The stacking ensemble achieves higher top-1 accuracy than Random Forest, linear SVM, and Multinomial Naive Bayes evaluated individually on the same hold-out set.
- **H2.** Applying academic affinity boosts changes the rank order for learners whose degree matches a different domain from the one favored by their skills, in the direction of the degree domain.
- **H3.** At least 80% of participating students rate the explanation of their skill gap as clear and useful (Section 3.4 defines the measure).

1.7 Scope, Limitations, and Assumptions

1.7.1 Scope

The component covers technical IT roles (more than forty role families, including software engineering, data and AI, cloud and DevOps, cybersecurity, quality assurance, business intelligence, and robotic process automation). Inputs are limited

to skills and degrees. Outputs are career rankings, readiness and gap indices, a learning roadmap, and a pivot graph.

1.7.2 Limitations

- Skills are self-reported and are matched by name; proficiency level is not modelled.
- The training corpus is derived from public job postings and curriculum-aligned synthetic data; it reflects employer demand, not graduate outcomes.
- Work experience, soft skills, certifications, and personality traits are outside the scope of the component.
- The component recommends role families and skills; it does not predict salary, employment probability, or individual suitability beyond skills and degree.

1.7.3 Assumptions

- The frequency with which a skill is requested for a role is a reasonable proxy for its market importance.
- Job postings in the corpus are representative of current employer expectations for the corresponding roles.
- Students can recognize and select their own skills from a controlled vocabulary with fuzzy auto complete.

1.8 Individual Contribution

The author is the sole designer and developer of the Career Pathway and Skill Gap Analyzer. The individual contributions are:

- design and implementation of the data acquisition and NLP pipeline (scrapers, cleaner, skill extractor, and synthetic enrichment generators);
- design, training, and evaluation of the stacking ensemble career recommender;
- design of the academic affinity cascade and the hierarchical qualification-tier resolver;
- design of the Neo4j graph schema, the batched ingestion script, and the SGI and Market Readiness formulations;

- implementation of the FastAPI microservice with typo-tolerant suggestion endpoints and the graph-data endpoint;
- implementation of the React front end (Analyzer, Roadmap, GraphExplorer, Dashboard) and its integration into the platform shell.

Contributions of other group members are acknowledged where the component interacts with them (Section 5.3) and are not claimed here.

1.9 Thesis Structure

The remainder of this thesis is organized as follows. Chapter 2 reviews related work and synthesizes the research gap into a conceptual framework. Chapter 3 describes the research methodology, data, evaluation measures, and ethical considerations. Chapter 4 presents the requirements and the design of the solution. Chapter 5 describes implementation, integration, and testing. Chapter 6 reports the results and validation. Chapter 7 discusses the findings, contributions, and threats to validity. Chapter 8 considers commercialization and sustainability. Chapter 9 concludes the thesis and outlines future work. References, a glossary, and appendices follow.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

The review follows a structured but non-exhaustive approach. Searches were carried out over IEEE Xplore, ACM Digital Library, ScienceDirect, SpringerLink, and Google Scholar using combinations of the terms *career recommendation*, *job recommender system*, *skill gap analysis*, *skill extraction from job postings*, *stacking ensemble*, *knowledge graph*, *explainable AI*, and *learning pathway*. Priority was given to peer-reviewed work, standards, and documentation of widely used tools. Sources were screened for relevance to three questions: how careers are predicted from skills, how gaps are measured, and how results are explained to the user. Industry platforms were reviewed from their public documentation and feature descriptions; no proprietary information was used.

The scope of the review is deliberately focused on the technical foundations used by this component (supervised text and tabular classification, ensemble learning, graph databases, and explainability) and on the application domain (career guidance and skill gap analysis). Recommender systems more broadly are covered only to the extent needed to position the work [3], [4].

2.2 Domain and Theoretical Foundations

2.2.1 Career guidance and the skills-based labor market

Career guidance has traditionally drawn on trait-and-factor and developmental theories, which match an individual's attributes to occupational requirements. Contemporary labor-market analyses emphasize skills rather than qualifications alone: the World Economic Forum reports that employers expect a substantial share of workers' core skills to change within a few years, and that technology-related skills are among the fastest growing [5]. For IT students this implies that guidance must be tied to current employer demand rather than to static curriculum descriptions.

Occupational taxonomies provide structure for this matching. O*NET describes occupations by tasks, skills, and knowledge for the United States labor market [1], and ESCO provides a multilingual classification of skills, competences, qualifications, and

occupations for Europe [2]. Both are valuable references, but neither captures the vocabulary, technology stacks, or degree expectations of Sri Lankan employers, which motivates the use of locally scraped postings in this research.

2.2.2 Text-derived features and TF-IDF weighting

Job postings are unstructured text. Skill extraction converts them into a structured set of skill tokens using dictionaries, pattern matching, and named-entity tools [6], [7]. Once represented as sets, skills can be encoded as binary vectors (one dimension per skill). Term frequency–inverse document frequency (TF-IDF) weighting down-weights tokens that occur in many documents and up-weights discriminative tokens [8]. In this component the "documents" are job postings, and the "terms" are skills, so ubiquitous skills such as *Git* or *SQL* receive less weight than distinctive skills such as *Kubernetes* or *PyTorch*. This directly addresses the sparsity and frequency-bias problems described in Section 1.3.1.

2.2.3 Supervised classifiers used in this work

Three complementary base learners are used. Random Forests combine many decorrelated decision trees and capture non-linear interactions between skills [9]. Support Vector Machines find maximum-margin separating hyperplanes and perform well in high-dimensional sparse spaces [10]; probability estimates can be obtained by Platt scaling [11]. Multinomial Naive Bayes is a fast probabilistic classifier for count-like features that assumes conditional independence [12]. Their errors are partly uncorrelated, which is the precondition for benefiting from ensembling [13], [14].

2.2.4 Stacked generalization

Stacked generalization trains a meta-learner on the out-of-fold predictions of a set of base learners so that the meta-learner learns which base learner to trust in which region of the input space [15]. Unlike simple voting, stacking can learn asymmetric trust: for instance, relying on the Random Forest for rare skill combinations and on the SVM for dense profiles. Using out-of-fold predictions via cross-validation prevents the meta-learner from being trained on predictions the base learners made on their own training data [16], [17]. The scikit-learn implementation of `StackingClassifier` is used [18].

2.2.5 Knowledge graphs and graph databases

A knowledge graph represents entities and typed relationships, allowing queries that traverse relationships [19]. Property-graph databases such as Neo4j store nodes and relationships with attributes and are queried with Cypher [20], [21]. For career data the natural entities are job roles, skills, and degrees, and the natural relationships are *requires skill* and *requires degree*, with the posting frequency stored as a relationship weight. Multi-hop traversals (role → skill → other roles) express career pivots directly, which is cumbersome in relational schemas.

2.2.6 Explainability and interpretability

Explainable AI (XAI) seeks to make model outputs understandable to humans [22], [23]. Post hoc methods such as LIME [24] and SHAP [25] approximate the behavior of an opaque model locally. An alternative, advocated for high-stakes decisions, is to make the decision pipeline interpretable by construction [26]. This research adopts a hybrid position: the classifier is statistical, but every number shown to the learner (readiness, gap, missing skills, and affinity boosts) is computed by transparent, documented formulas over graph evidence, so that explanation does not depend on approximating the classifier.

2.3 Related Research

2.3.1 Career and job recommender systems

Recommender-system research distinguishes content-based, collaborative, knowledge-based, and hybrid approaches [3], [4]. Collaborative filtering requires interaction histories that a new student does not have (the cold-start problem), so content-based and knowledge-based approaches dominate in career guidance. Content-based job recommenders match candidate profiles with postings through keyword overlap, cosine similarity on TF-IDF vectors, or supervised classifiers. Supervised classification of a profile into a role family is a pragmatic formulation that provides calibrated class probabilities, which can be used to produce top-k lists.

2.3.2 Skill extraction and gap analysis

Skill gap analysis is commonly implemented as set difference between a candidate's skills and the skills of a target occupation. This is simple but treats all skills as equally important. Weighted approaches assign importance by frequency or by expert judgement. The present work adopts frequency-derived weights computed from the graph and normalized to a 1–10 scale per role, and extends the gap to include a degree component.

2.3.3 Ensemble learning for classification

Ensembles have been shown to reduce variance and improve generalization relative to their members when the members are accurate and diverse [13], [14]. Stacking is among the more flexible ensemble forms, at the cost of training complexity. Evaluation of multi-class classifiers should report more than accuracy: macro-averaged measures treat each class equally, while weighted measures reflect class prevalence [27]. For recommendation, top-k accuracy is also meaningful because users examine a short list of alternatives rather than a single prediction.

2.3.4 Graph-based and explainable recommendation

Knowledge graphs have been used to enrich recommendation with side information and to produce path-based explanations [19]. In career contexts a path such as *Python → Machine Learning Engineer → Computer Vision Engineer* communicates why a skill matters beyond the immediate target role. The pivot analysis in this component generalizes this idea by showing, for each missing skill, which other roles it unlocks.

2.4 Existing Systems, Products, and Techniques

Several categories of products address parts of the problem. General job portals, including the Sri Lankan portal TopJobs [28], expose postings but do not analyze a student's profile. Occupational information services such as O*NET OnLine [1] offer detailed role descriptions and skill lists, and allow searching occupations by skills, but they are not personalized to a degree and are US-centered. Learning-roadmap websites present static diagrams of what to learn for a given role; they do not adapt to what the learner already knows. Commercial professional networks provide "skills you are

missing" suggestions for specific postings but do not explain the weighting and are not designed for undergraduates choosing among role families. Table 2.1 compares these categories with the proposed component along the dimensions that matter to this research.

Table 2.1: Comparison of existing solution categories with the proposed component

Dimension	Job portals	Occupation databases	Static roadmaps	Network skill hints	This component
Personalized to skills	No	Partial	No	Yes	Yes
Considers degree	No	No	No	Partial	Yes (tiered)
Quantified gap/readiness	No	No	No	Partial	Yes (SGI)
Local market evidence	Yes	No	No	Partial	Yes
Transparent weighting	n/a	Partial	n/a	No	Yes
Career pivot exploration	No	Partial	No	No	Yes (graph)

The comparison is qualitative and based on publicly documented features; it is not a benchmark of accuracy. It shows that no single category offers the combination of personalization, degree awareness, quantified gap, local evidence, transparency, and pivot exploration.

2.5 Gap Synthesis and Conceptual Framework

Literature supports three design decisions. First, a TF-IDF weighted stacking ensemble is an appropriate and well-founded classifier for sparse skill vectors (Gap G1). Second, graph-structured evidence permits a transparent, reproducible gap measure and pivot analysis (Gaps G2 and G4). Third, academic background should be modelled explicitly as a calibrated signal and as a tiered qualification, rather than being ignored or treated as binary (Gap G5). Local market grounding addresses Gap G3. Figure 2.1 shows the conceptual framework that results.

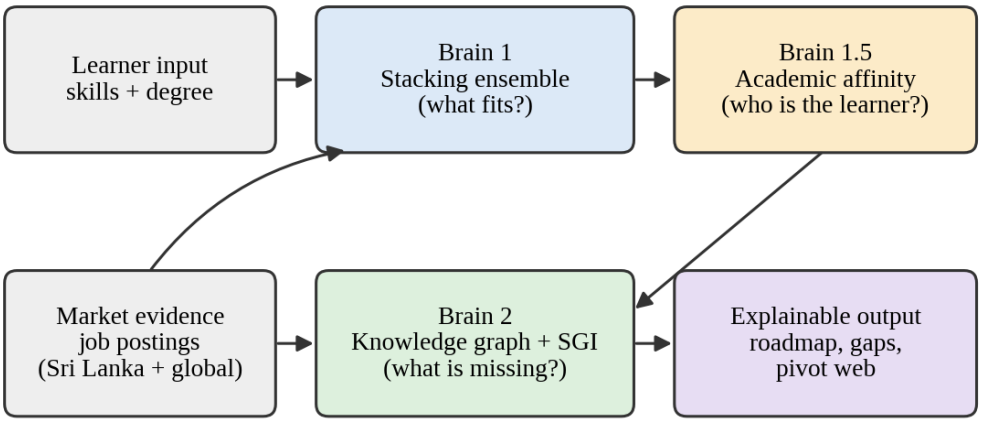


Figure 2.1: Conceptual framework of the dual-engine career pathway analysis

In the framework, Brain 1 answers *what fits the learner's skills*, Brain 1.5 answers *who the learner is academically*, and Brain 2 answers *what is missing and what else becomes possible*. Market evidence feeds both the classifier (as training data) and the graph (as weights), which keeps prediction and explanation consistent. The framework is evaluated against the research questions in Chapters 3 and 6.

3 RESEARCH METHODOLOGY

3.1 Research Design

The research follows the design-science paradigm, in which a purposeful artefact is built to address an identified problem and is rigorously evaluated [29]. The artefact is the Career Pathway and Skill Gap Analyzer. The research is mixed-methods: quantitative evaluation of the classifier on a held-out set of job-posting records, deterministic verification of the gap formulas, and a user study that combines standardised usability scales with open-ended feedback. Figure 3.1 summarizes the research cycle.

Design-science research cycle: build, evaluate, refine

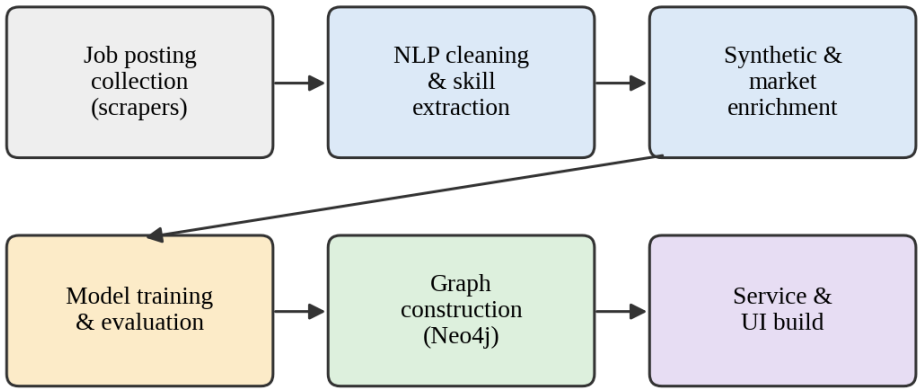


Figure 3.1: Research and development cycle of the component

Development was iterative. The data pipeline, classifier, graph, and gap engine were first built in that order; evaluation results then fed back into enrichment of the training data and into the academic affinity design, which was added after informal testing exposed the Data Science Degree Trap (Section 1.3.1).

3.2 Data, Participants, and Study Materials

3.2.1 Job-posting corpus

The primary data are job postings for IT roles. Raw postings were collected with two scrapers: `topjobs_scraper.py`, which uses Selenium with headless Chrome and

BeautifulSoup to read the Sri Lankan TopJobs portal [28], and `multi_role_scraper.py`, a multi-portal extractor. Raw text was stored under `data/raw` and `raw_multi_role_jobs.csv`. The NLP cleaner (`multi_role_nlp_cleaner.py`) then produced the master training dataset `cleaned_multi_role_jobs.csv`, in which each record contains a role label, a list of extracted skills, and, where present, a degree requirement. Table 3.1 lists the data artefacts.

Table 3.1: Data artefacts used by the component

Artefact	Description	Size / coverage
raw_multi_role_jobs.csv	Raw scraped postings (unprocessed text)	5000+ postings
cleaned_multi_role_jobs.csv	Master training dataset: role, skills list, degree	200+ roles; 600+ skills
delta_new_jobs.csv	Synthetic and enriched job delta aligned to curricula and the Sri Lankan market	2000+ records
Skill dictionary	Categorized IT skills used by the extractor	350+ skills
Neo4j graph	Job Role, Skill and Degree nodes with weighted relationships	200+ roles; 600+ skills; 4000+ relationships

3.2.2 Data enrichment

Scraped postings alone under-represent some roles and some degree–skill combinations. Three generators were therefore written: `generate_synthetic_delta.py` (curriculum-aligned records), `generate_sl_market_enrichment.py` (Sri Lankan market demand), and `generate_sl_business_delta.py` (business and domain expansion). Synthetic records are used to balance coverage and are a source of validity risk, which is discussed in Section 7.5.

3.2.3 Participants

User validation involves undergraduate students from IT and Business-related degree programs at SLIIT. Participation was voluntary and anonymous (Section 3.6).

3.3 Experimental and Study Procedure

3.3.1 Model training and evaluation procedure

1. Load `cleaned_multi_role_jobs.csv` and parse the stringified skill lists with `ast.literal_eval`; discard rows with no skills.
2. Encode skills with `MultiLabelBinarizer` and weight them with `TfidfTransformer` (Section 4.4.1).
3. Split the data 80% / 20% into training and test sets, stratified by role, with `random_state=42`.
4. Train a `StackingClassifier` with three base learners and a Logistic Regression meta-learner using 5-fold cross-validation for the out-of-fold predictions.
5. Train each base learner individually on the same split to obtain baselines.
6. Evaluate on the untouched test set using the measures in Section 3.4 and serialize the fitted artefacts with `joblib`.

Table 3.2: Model configuration

Component	Configuration
Random Forest	200 trees, unlimited depth, class_weight = balanced, random_state = 42
Support Vector Classifier	Linear kernel, probability = True (Platt scaling), class_weight = balanced, random_state = 42
Multinomial Naive Bayes	Default smoothing
Meta-learner	Logistic Regression, max_iter = 1000, class_weight = balanced
Stacking	cv = 5 (out-of-fold probabilities passed to the meta-learner)
Split	80% train / 20% test, stratified, random_state = 42

3.3.2 Verification procedure for the gap engine

The Skill Gap Index and academic affinity logic are deterministic. They are verified by constructing test profiles whose expected outcomes can be calculated by hand from the formulas in Section 4.4 and comparing them with the engine's output (Section 5.4).

3.3.3 User validation procedure

Each participant completes a scripted session of about twenty minutes: (i) a short briefing; (ii) three tasks on the analyzer (enter skills and degree, interpret the top-three careers, and identify the first skill to learn for one role); (iii) exploration of the roadmap and the graph explorer; and (iv) a questionnaire. The questionnaire (Appendix A) contains the ten-item System Usability Scale [30], items on perceived usefulness and ease of use adapted from the Technology Acceptance Model [31], items on clarity of explanation, and open questions.

3.4 Measures and Evaluation Metrics

Classification quality is measured with top-k accuracy, the proportion of test records whose true role appears among the k highest-probability predictions:

$$Top\text{-}k = (1/N)\ \Sigma\ 1[\ y_i \in \hat{y}_i(k)\ ] \tag{3.1}$$

and with precision, recall, and F1-score for each role, summarized by macro and weighted averages [27]:

$$F1 = 2 \cdot P \cdot R / (P + R) \tag{3.2}$$

The macro average weights every role equally and therefore exposes weak performance on rare roles; the weighted average weights roles by their support. Table 3.3 maps each research question to its measures.

Table 3.3: Mapping of research questions to evaluation measures

RQ	Measure	Success criterion
RQ1 / H1	Top-1, top-3, top-5 accuracy; macro and weighted F1; comparison with each base learner on the same split	Stacking top-1 accuracy exceeds every single base learner
RQ2 / H2	Rank change of roles under affinity boosts on constructed degree–skill profiles	Roles matching the degree domain rise in rank
RQ3 / H3	SUS score; Likert items on clarity and usefulness; task completion	SUS above 68 (industry average); at least 80% positive on clarity

The SUS benchmark of 68 is the commonly cited average score for the instrument [30]. Usability is additionally interpreted using Nielsen's attributes of learnability,

efficiency, memorability, errors, and satisfaction [32]. System responsiveness is reported as end-to-end latency of the analysis endpoint.

3.5 Data Analysis

Classification results are computed on a single stratified hold-out split and are reported as point estimates; k-fold cross-validation inside the stacking procedure provides the out-of-fold predictions but is not used for the reported test metrics [16]. Per-role precision, recall, and F1 are reported in Appendix A. Questionnaire data are summarized with descriptive statistics (mean, standard deviation, and frequency). The SUS score is calculated using the standard scoring procedure and, given a small sample, is reported with a confidence interval where the sample permits. Open-ended responses are coded thematically by the author. Where participants differ by degree program, group differences are reported descriptively, and no inferential claim is made.

3.6 Ethics, Privacy, Security, and Responsible AI

- **Data sources.** Only publicly accessible postings were collected, at a modest request rate, and only fields relevant to the research (role, skills, degree requirement) were retained. No personal data about applicants or recruiters were stored.
- **User data.** The component asks for skills and degree only. No names, identifiers, or contact details are needed to obtain an analysis. For user validation, participants gave informed consent, were told that participation is voluntary, and could withdraw at any time. Responses are anonymized, in line with the principles of the Sri Lankan Personal Data Protection Act [33].
- **Security.** Database credentials are kept in an `.env` file excluded from version control; the graph is accessed over TLS; request bodies are validated with Pydantic; and the main web-application risks listed by OWASP [34] were considered in the design (Section 4.5).
- **Responsible AI.** The component presents recommendations as decision support, shows the evidence behind each number, and does not make

automated decisions about the student. Its dependence on the job-market sample and on synthetic data is disclosed. Bias is a recognized risk: roles with few postings may be recommended less often, and degree boosts could favor students from certain programs. These risks are discussed in Chapter 7. The transparency approach is consistent with the transparency obligations that the EU AI Act places on systems that interact with people [35], although the component is a low-risk educational aid.

3.7 Reproducibility and Research Artefacts

All artefacts are in the project repository under `R26-IT-073`. Random seeds are fixed (`random_state=42`), library versions are pinned in `ai-engine/requirements.txt`, the trained model, skill encoder, and TF-IDF transformer are serialised as `stacking_ensemble_model.pkl`, `skill_encoder.pkl`, and `tfidf_transformer.pkl`, and the graph is rebuilt from the cleaned dataset by `graph_builder.py`. A launcher (`start_career_ai.bat`) starts the service. Section 5.5 and Appendix A list the commands needed to reproduce the results. The raw scraped postings may be subject to the terms of the source sites and are therefore not redistributed with this thesis; the cleaned feature table and scripts are sufficient to reproduce the model.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

Requirements were derived from the problem statement (Section 1.3), from informal discussions with fellow students, and from the platform-level needs of the StudyFyX project and are expressed in the style recommended by ISO/IEC/IEEE 29148 [36]. Table 4.1 lists the functional requirements and Table 4.2 the non-functional requirements, classified using the quality characteristics of ISO/IEC 25010 [37].

Table 4.1: Functional requirements

ID	Requirement	Objective
FR1	The system shall accept a set of skills with typo-tolerant auto-complete from the corpus vocabulary.	O1, O6
FR2	The system shall accept a degree with shorthand expansion (for example bsc → bachelor, se → software engineering).	O3, O6
FR3	The system shall return the top three career matches with confidence scores.	O2
FR4	The system shall adjust confidence using the learner's degree and re-rank results.	O3
FR5	The system shall compute market readiness and SGI for each recommended role.	O4
FR6	The system shall list missing skills and degree requirements ordered by market weight.	O4
FR7	The system shall generate a phased learning roadmap for a selected role.	O6
FR8	The system shall generate a career pivot graph showing roles unlocked by missing skills.	O5
FR9	The system shall provide a dashboard summarizing readiness and allow exporting a PDF summary.	O6

Table 4.2: Non-functional requirements

ID	Quality attribute	Requirement
NFR1	Performance efficiency	Cached analysis responses should be returned in under 50 ms; uncached responses within a few seconds.
NFR2	Usability	A first-time user shall be able to obtain an analysis without instructions; target SUS above 68.
NFR3	Reliability	The service shall expose a health endpoint and shall return structured errors for invalid input.
NFR4	Security	Credentials shall not be stored in source code; input shall be validated; transport to the database shall be encrypted.
NFR5	Maintainability	Classifier, graph, and API logic shall be separated into modules with documented interfaces.
NFR6	Compatibility	The service shall interoperate with the shared StudyFyX front end through JSON over HTTP.
NFR7	Explainability	Every score presented to the user shall be traceable to documented weights and rules.

4.2 Component Architecture within the Complete Solution

The component follows a layered, service-oriented architecture (Figure 4.1). The React front end communicates with the FastAPI gateway over REST [38], [39]. The gateway orchestrates the three analytical stages and returns a single JSON document per request. Heavy artefacts (the serialized model, encoder, and transformer) are loaded once at start-up, and responses to repeated requests are held in an in-memory cache. The graph database is the only external dependency.

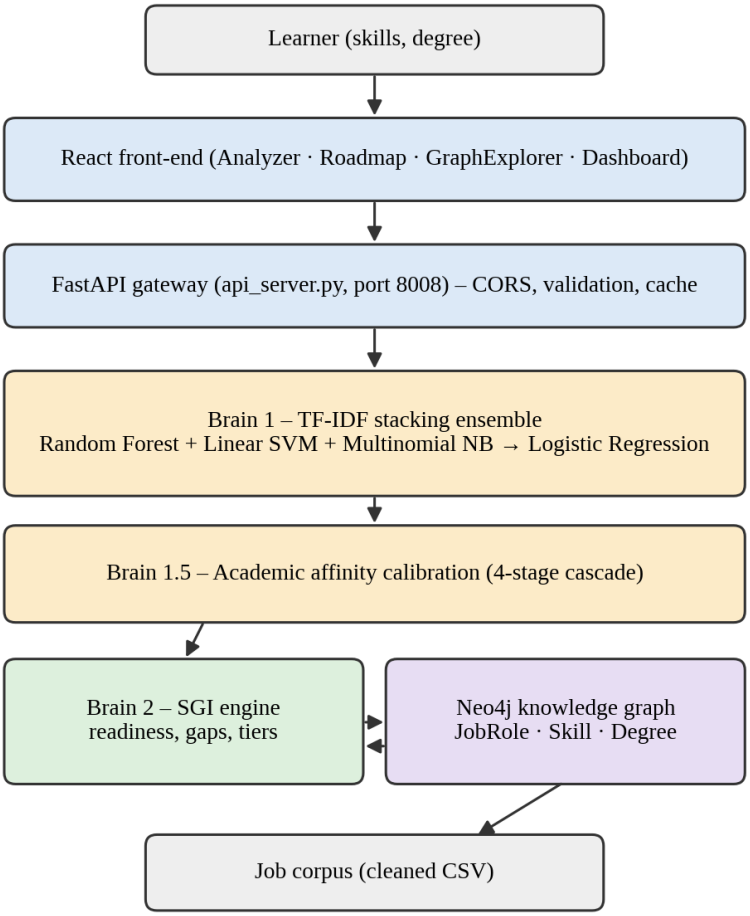


Figure 4.1: Layered architecture of the Career Pathway and Skill Gap Analyzer

Within the wider platform, the component is deployed as an independent microservice on port 8008 and its React pages are mounted inside the shared StudyFyX-`app` shell under the `Member2` module. Other components do not call the service directly; they link to it through the shell's navigation. This loose coupling allows each member's service to evolve independently (Section 5.3).

4.3 Design Alternatives and Decisions

Several design choices were made after considering alternatives. Table 4.3 summarizes the principal decisions and the reasons for them.

Table 4.3: Design alternatives and decisions

Decision area	Alternatives considered	Decision and rationale
Classifier	Single Random Forest; single SVM; Naive Bayes; neural network	Stacking ensemble. Base models are diverse and complementary, and the meta-learner learns when to trust each; a neural network was rejected because the data are small, sparse, and need to remain interpretable.
Feature weighting	Binary skills; raw counts; TF-IDF	TF-IDF over binary skills, to reduce the dominance of ubiquitous skills.
Knowledge store	Relational tables; JSON files; Neo4j	Neo4j. Role→skill→role traversals for pivots are natural in Cypher and costly in SQL joins.
Degree handling	Ignore; binary match; tiered match with affinity	Tiered resolver plus four-stage affinity cascade, which is deterministic and explainable.
Gap measure	Set difference; cosine distance; weighted index	Weighted index, so that high-demand missing skills dominate the gap.
Explanation	Post hoc LIME or SHAP; interpretable pipeline	Interpretable pipeline for the gap and affinity layers; the classifier is not explained post hoc.
API style	Flask; Django REST; FastAPI	FastAPI: asynchronous, type-validated, and self-documenting through OpenAPI.
Front end	Server-rendered pages; React SPA	React SPA, consistent with the shared platform shell.

4.4 Detailed Design

4.4.1 Feature representation

Let S be the set of all skills in the training corpus and D the set of postings. Each posting is encoded as a binary vector over S by the multi-label binarizer. The transformer then re-weights each entry with the TF-IDF scheme [8]:

$$w(s, d) = tf(s, d) \cdot idf(s), \quad idf(s) = ln[(1 + |D|) / (1 + df(s))] + 1 \qquad (4.1)$$

where $df(s)$ is the number of postings that mention skill s. The scikit-learn implementation applies L2 normalization to each vector [18]. The result is the matrix X used for training. Because the same transformer is applied to a student's skills at prediction time, student vectors and posting vectors lie in the same space.

4.4.2 Brain 1: stacking ensemble

The ensemble comprises a Random Forest, a linear SVC, and a Multinomial Naive Bayes classifier as base learners and a class-balanced Logistic Regression as meta-learner (Table 3.2). During training, five-fold cross-validation produces out-of-fold class-probability vectors from each base learner; these are concatenated and used to

train the meta-learner [15]. At prediction time the base learners run on the student's vector, and the meta-learner returns a probability for every role. The method `predict_top_pathways` sorts roles by probability and returns the top *n* (default three) as role–confidence pairs, with confidence expressed as a percentage.

4.4.3 Knowledge graph schema and weights

The graph contains three node labels and two relationship types (Figure 4.2). `REQUIRES_SKILL` relationships carry a `weight` equal to the number of postings in which the role and skill co-occur; `REQUIRES_DEGREE` relationships carry a market weight for the degree requirement. The batch ingestion script `graph_builder.py` loads postings in batches of 500 with `UNWIND` and `MERGE`, creating relationships with weight 1 and incrementing the weight on every further co-occurrence.

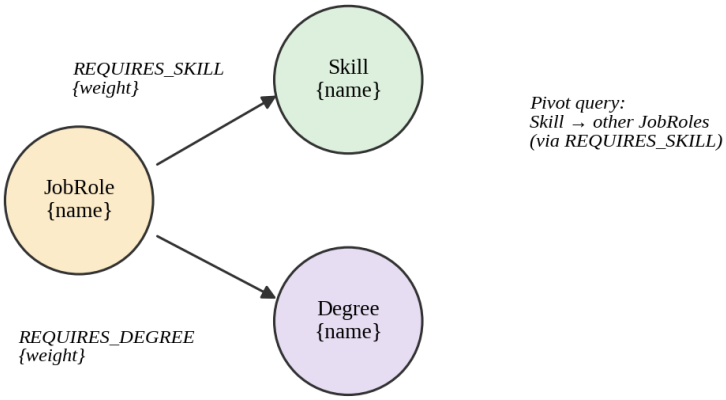


Figure 4.2: Schema of the Neo4j knowledge graph

For a target role, the engine retrieves the ten most heavily weighted skills, and normalizes each weight to a 1–10 scale relative to the most frequently requested skill for that role:

$$W(s) = 10 \cdot weight(role, s) / max[\ weight(role, k) \text{ over all skills } k \text{ of the role }\ ] \quad (4.2)$$

This relative scale makes weights comparable across roles with different numbers of postings. The Cypher query that performs the retrieval and normalisation is shown below.

```
MATCH (r:JobRole {name: $role})-[rel:REQUIRES_SKILL]->(s:Skill)
```

```
WITH r, rel, s
MATCH (r)-[all_rel:REQUIRES_SKILL]->()
WITH rel, s, max(all_rel.weight) AS max_weight
RETURN s.name AS skill,
       (toFloat(rel.weight) / coalesce(max_weight, 1.0)) * 10.0 AS weight
ORDER BY weight DESC LIMIT $limit
```

For roles that are portfolio-centred (UI/UX Designer, Frontend Developer, Web Developer, and Game Developer) the academic benchmark is relaxed: the recommended degree is reported as "CS Degree OR Strong Portfolio" and the degree weight is reduced to 2.0, instead of the usual 5.0 to 10.0. This *Creative Tech Override* reflects the fact that employers in these fields often accept demonstrable work in place of a formal qualification.

4.4.4 Brain 1.5: academic affinity cascade

After the classifier ranks roles, the affinity stage compares the student's degree with each role and adds a boost to the confidence of roles that the degree supports. The stages are tried in order and the first that applies determines the boost (Figure 4.3):

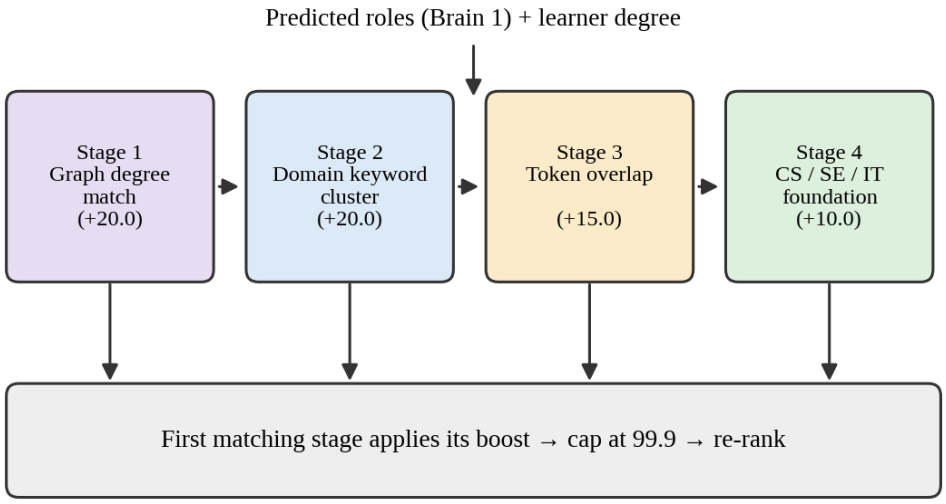


Figure 4.3: Four-stage academic affinity cascade

1. **Graph match (+20.0).** The graph contains a `REQUIRES_DEGREE` relationship from the role to a degree that matches the student's degree.

2. **Domain keyword cluster (+20.0).** The degree and the role fall in the same one of six IT verticals: Data and AI, Web Development, Cloud and DevOps, Cybersecurity, Quality Assurance, and Management.
3. **Token overlap (+15.0).** After removing academic stop-words (such as *bsc*, *bachelor*, *msc*, *science*, *honours*), the remaining root terms of the degree overlap with the role name.
4. **Foundational boost (+10.0).** The degree mentions Computer Science or Software Engineering, which supports IT roles generally.

$$c'(r) = min(99.9,\ c(r) + \beta(r, degree)) \quad (4.3)$$

Here $c(r)$ is the classifier confidence for role r and β is the boost from the first matching stage. The adjusted list is re-sorted. As an illustration, if the classifier gives Web Developer 62.0, Data Scientist 55.0, and Data Analyst 48.0 for a student with a degree in Data Science who lists mostly basic web skills, the data roles receive +20.0 and become 75.0 and 68.0, while the web role remains at 62.0 unless the degree also matches it. The ranking changes from Web Developer first to Data Scientist first, which is the correction sought. The example is constructed to explain the mechanism; measured behavior is reported in Section 6.1.3.

4.4.5 Hierarchical qualification tiers

Whether a student's qualification satisfies a role's requirement is determined by comparing tiers rather than by string equality. Degree strings are normalized to tokens, which are mapped to the tier values in Table 4.4.

Table 4.4: Qualification tier hierarchy

Tier	Qualification tokens	Outcome versus the requirement
4	phd, doctorate	Higher tier: 120% of degree weight
3	master, masters, msc, mba, ma	Higher tier: 120% of degree weight
2	bachelor, bachelors, bsc, ba, degree	Equal tier: 100% of degree weight
1	hnd, diploma	Lower tier: 0%, degree listed as missing
0	certificate	Lower tier: 0%, degree listed as missing

The outcome in the third column is shown for a role requiring a bachelor's degree. For example, a Master's holder applying to a role requiring a bachelor's degree receives 120% of the degree weight, whereas a diploma holder receives none and the missing degree is added to the list of missing requirements.

4.4.6 Brain 2: Skill Gap Index and market readiness

For each role the engine computes the total market weight as the sum of the weights of the ten core skills and the degree weight, and the user's matched weight as the sum of the weights of the core skills the user possesses plus the awarded degree weight:

$$TMW = \Sigma\, W(s)\ \ [s \in Core]\ \ +\ \ W(deg) \tag{4.4}$$

$$UAW = \Sigma\, W(s)\ \ [s \in Core \cap User]\ \ +\ \ W(awarded) \tag{4.5}$$

$$Readiness\ (\%) = min(\ 100,\ \ round(\ UAW\ /\ TMW \times 100,\ 1\ )\ ) \tag{4.6}$$

$$SGI = max(\ 0,\ \ round(\ 100 - Readiness,\ 1\ )\ ) \tag{4.7}$$

The cap in the readiness formula is needed because an exceeded qualification can award 120% of the degree weight while the denominator contains only 100%. A worked example with ten core skills with weights 10, 9.1, 8.5, 8.0, 7.8, 7.2, 6.5, 6.2, 5.5, and 5.0 (total 73.8) and a degree weight of 5.0 (so TMW = 78.8) is given in Table 4.5. The student owns the skills weighted 10, 9.1, 8.0, 7.2, and 6.5 (matched weight 40.8). The values are illustrative and are chosen only to show the arithmetic.

Table 4.5: Worked example of readiness and SGI under three degree outcomes (illustrative)

Degree outcome	Awarded degree weight	UAW	Readiness (%)	SGI
Tier met (bachelor vs bachelor)	5.0	45.8	58.1	41.9
Tier exceeded (master vs bachelor)	6.0	46.8	59.4	40.6
Tier not met (diploma vs bachelor)	0.0	40.8	51.8	48.2

Source: Calculated from Equations 4.4 to 4.7 with the illustrative weights given in the text

Missing requirements are the core skills the student does not have, plus the degree if the tier is not met, each reported with its weight and ordered from highest to lowest. This ordering is what drives the roadmap: the skill with the highest weight is the one that closes the largest part of the gap.

4.4.7 Career pivot (Synergy Web)

The graph-data endpoint produces the Synergy Web (Figure 4.4). The center node is the target role. Surrounding nodes are its ten core skills, colored green when the student already has the skill and amber when missing. A further Cypher query finds, for each skill, other roles that also require it with high weight; these appear as purple pivot nodes connected by an `UNLOCKS` edge. A student can thus see, for example, that learning PyTorch for a Data Scientist role also moves them toward a Computer Vision Engineer role.

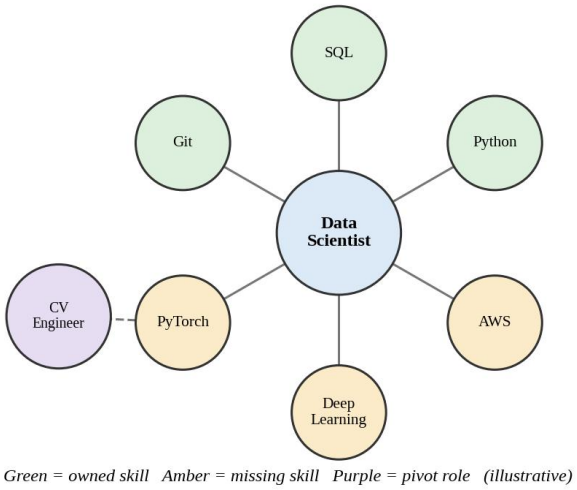


Figure 4.4: Structure of the Skill Synergy and Career Pivot Web (illustrative)

4.4.8 Service interface

The FastAPI service exposes the endpoints in Table 4.6. Requests and responses are JSON documents validated with Pydantic. Suggestion endpoints combine substring matching with `difflib.get_close_matches` (cut-off 0.55), which ranks candidates by the Ratcliff–Obershelp similarity ratio [40]; this tolerates misspellings such as “pyhton”.

Table 4.6: REST endpoints of the career AI service

Endpoint	Method	Purpose
/api/health	GET	Health status and engine load indicator
/api/suggest-skills?q=	GET	Typo-tolerant skill autocomplete over the corpus vocabulary
/api/suggest-degrees?q=	GET	Degree autocomplete with shorthand expansion
/api/generate-top3	POST	Top three careers with confidence, readiness, SGI, recommended degree, and missing requirements
/api/generate-pathway	POST	Legacy endpoint returning the single top pathway
/api/graph-data?role=&user_skills=	GET	Nodes and links for the Synergy Web

4.4.9 User interface design

The interface comprises five pages. *Landing* introduces the tool. *Analyzer* has two columns: a skill chip input with auto-complete, a degree selector, and quick-add pills

on the left, and a live analysis card with the top three careers, radial readiness meters, and missing-skill badges on the right. *Roadmap* presents the plan in three phases: Phase 1 Academic and Prerequisite Foundations, Phase 2 High-Impact Core Competencies ordered by market weight, and Phase 3 Industry Toolchains and Specializations; the student can switch between the three roles to compare the effort. *GraphExplorer* renders the force-directed Synergy Web. *Dashboard* summarizes readiness, market demand cards, and offers a PDF export. State is held in `CareerContext` (profile, top pathways, selected pathway, graph data, loading and error state) and the theme in `ThemeContext` (dark and light). Design follows basic usability heuristics [32]: visible system status (loading indicators), match with the real world (industry vocabulary), error prevention (autocomplete), and recognition rather than recall (chips and quick-add pills).

4.5 Standards and Professional Practice

Table 4.7: Standards and good practice applied

Standard / practice	Application in this component
ISO/IEC/IEEE 29148 [36]	Structure and wording of functional and non-functional requirements
ISO/IEC 25010 [37]	Quality attributes used to classify non-functional requirements and testing
REST [38] and OpenAPI	Resource-oriented endpoints; automatic API documentation at /docs
OWASP Top 10 [34]	Input validation, secret management, restricted CORS configuration, and dependency hygiene
Sri Lanka PDPA [33]	Data minimization: no personal identifiers are required or stored
IEEE reference style	Citation of sources in this thesis
Version control and modular code	Separation of NLP, models, API, and UI layers; reproducible launch script

5 IMPLEMENTATION, INTEGRATION, AND TESTING

5.1 Development Environment and Technology Stack

The component was developed on Windows using Python virtual environments and Node.js. The technology stack is summarized in Table 5.1. Each technology was selected for a specific role in the design (Section 4.3).

Table 5.1: Technology stack

Layer	Technology	Role
Language	Python 3.10+ (3.12 used); JavaScript (ES2022)	Backend and AI; front end
Web service	FastAPI, Uvicorn, Pydantic v2	Asynchronous REST API and validation [39]
Machine learning	scikit-learn (StackingClassifier, RandomForest, SVC, MultinomialNB, LogisticRegression)	Brain 1 [18]
Feature engineering	MultiLabelBinarizer, TfidfTransformer, pandas, NumPy, joblib	Encoding, weighting, serialization
NLP	spaCy (en_core_web_sm), regular expressions	Skill and degree extraction [6]
Graph database	Neo4j AuraDB and the neo4j Python driver; Cypher	Knowledge graph [21]
Scraping	Selenium (headless Chrome), BeautifulSoup4, Requests	Job-posting acquisition
Fuzzy matching	difflib	Typo-tolerant suggestions
Front end	React 18+, Vite, framer-motion, lucide-react, axios, CSS3	User interface
Visualization	Force-directed graph (Canvas/SVG), SVG metric rings	Synergy Web and readiness meters

5.2 Component Implementation

5.2.1 Data acquisition and NLP pipeline

The scrapers collect posting text with Selenium for portals that render content dynamically and with BeautifulSoup for static pages. `multi_role_nlp_cleaner.py` loads `en_core_web_sm`, normalises the text, and extracts skills by matching tokens and phrases against a dictionary of more than 150 categorized skills. The dictionary groups skills into programming languages, front-end technologies, cloud and DevOps, data and AI, security and quality assurance, and RPA and business intelligence. Degree

requirements are captured with regular expressions that recognize forms such as *B.Sc.*, *Master's*, and *Diploma*. Records with no extracted skills are removed, and the result is saved to `cleaned_multi_role_jobs.csv`.

5.2.2 Model training

`AdaptiveCareerRecommender` encapsulates training and prediction. `_prepare_training_matrix` parses the skill lists, fits the binarizer and the TF-IDF transformer, and persists both. `train_model` performs the stratified split, builds the stacking classifier, evaluates top-1, top-3, and top-5 accuracy and the classification report, and persists the model. The core definition is as follows.

```
base_learners = [
  ('rf',  RandomForestClassifier(n_estimators=200, max_depth=None,
                         random_state=42, class_weight='balanced')),
  ('svc', SVC(kernel='linear', probability=True, random_state=42,
              class_weight='balanced')),
  ('nb',  MultinomialNB())
]
meta = LogisticRegression(max_iter=1000, class_weight='balanced')
model = StackingClassifier(estimators=base_learners,
                           final_estimator=meta, cv=5)
```

`predict_top_pathways` loads the artefacts lazily, transforms the user's skills with the stored binarizer and transformer, calls `predict_proba`, and returns the top roles with confidence percentages.

5.2.3 Graph construction

`graph_builder.py` reads the cleaned dataset and writes it to Neo4j in batches of 500 rows using `UNWIND`, `MERGE`, and `ON CREATE`/`ON MATCH` clauses (Section 4.4.3). Batching avoids timeouts on the cloud-hosted instance and makes the load idempotent for nodes, because `MERGE` does not create duplicates.

5.2.4 SGI engine

`UnifiedAIEngine` in `sgi_engine.py` connects to Neo4j using credentials from the environment (`NEO4J_URI`, `NEO4J_USERNAME`, `NEO4J_PASSWORD`), and instantiates the recommender. Its main methods are `get_core_competencies`, `get_academic_benchmark` (including the Creative Tech Override), `apply_academic_affinity`, `evaluate_academic_profile`, and

_analyze_single_role, which evaluates Equations 4.4 to 4.7 and returns the readiness, SGI, recommended degree, and missing requirements for one role.

5.2.5 API server

api_server.py creates the FastAPI application with CORS middleware, loads the engine once at start-up, and implements the endpoints in Table 4.6. Computed analyses are stored in an in-memory cache keyed by the normalized skills and degree, so that repeated requests return without recomputation; the project target for cached responses is under 50 ms (NFR1). The graph-data endpoint assembles nodes into the groups target_role, skill_owned, skill_missing, and pivot_role, with links of type REQUIRES and UNLOCKS.

5.2.6 Front-end application

The front end is implemented as an isolated module (StudyFyX-app/Member2) with its own router entry point (index.jsx), styles (index.css), and API configuration (config/api.js, an Axios client). SkillInput.jsx implements the chip input with debounced calls to the suggestion endpoint, and DegreeInput.jsx the degree selector with alias expansion. Roadmap.jsx converts the missing requirements to the three phases. GraphExplorer.jsx runs a force-directed layout of the nodes and links returned by the API. Dashboard.jsx provides the readiness summary and PDF export.

5.3 Integration with the Other Components

Integration operates at two levels. At the **user-interface level**, the Member 2 module is mounted in the shared StudyFyX-app React shell, which supplies the navigation and the global layout and theme conventions used by all components. At the **service level**, the career service is a separate process on port 8008, so it can be started and tested independently with start_career_ai.bat. The service has no runtime dependency on the other components' back ends, and its only external dependency is Neo4j. Table 5.2 lists the integration points.

Table 5.2: Integration points with the platform

Integration point	Mechanism	Contract
Application shell	Route registered under the Member2 module in StudyFyX-app	Shared layout and theme tokens
Career service	HTTP/JSON over REST on port 8008	Endpoints in Table 4.6; CORS enabled for the shell origin
Explainability conventions	Each score is returned with the weights and rules that produced it	Fields missing_requirements, sgi_score, market_readiness

5.4 Verification and Software Testing

Testing is organized in four levels. The strategy and the quality characteristics of each level addresses are shown in Table 5.3.

Table 5.3: Test levels

Level	Scope	Quality characteristic
Unit	Skill extraction; tier resolver; affinity stages; SGI arithmetic; degree alias expansion	Functional correctness
Model	Hold-out evaluation; baseline comparison	Functional suitability (accuracy)
API / integration	Endpoints, validation errors, graph queries, cache behaviour	Reliability, compatibility
System / UI	End-to-end flows in the browser; usability sessions	Usability, performance efficiency

Table 5.4 lists the principal test cases. Expected outcomes are derived from the specification (Chapter 4) and, for arithmetic cases, from the worked example in Table 4.5. The Result column is completed when the tests are executed.

Table 5.4: Principal test cases

ID	Test	Input	Expected outcome	Result
T01	Tier met	User: BSc; role requires bachelor	Degree weight awarded 100%	pass
T02	Tier exceeded	User: MSc; role requires bachelor	120% of degree weight awarded	pass
T03	Tier not met	User: Diploma; role requires bachelor	0% awarded; degree listed missing	pass
T04	Readiness cap	User owns all core skills; MSc vs bachelor	Readiness = 100.0; SGI = 0.0	pass
T05	Worked example	Weights of Table 4.5; degree met	Readiness 58.1; SGI 41.9	pass

ID	Test	Input	Expected outcome	Result
T06	Affinity stage 1	Degree with a graph REQUIRES_DEGREE match	+20.0 to matching roles	pass
T07	Affinity cap	Confidence 95 with +20 boost	Result 99.9	pass
T08	Foundational boost	BSc Software Engineering; no other match	+10.0 to roles	pass
T09	Creative override	Role: Web Developer	Degree weight 2.0; degree text shows portfolio option	pass
T10	Typo tolerance	q = “pyhton”	Python in suggestions	pass
T11	Degree alias	q = “bsc se”	Expands to bachelor software engineering	pass
T12	Top-3 contract	POST /api/generate-top3 with valid body	Three roles; required fields present; sorted by confidence	pass
T13	Invalid input	Empty skills list	Structured error; no server fault	pass
T14	Graph data	GET /api/graph-data for a role	One target node; owned/missing groups correct; pivot nodes present	pass
T15	Cache	Repeat identical request	Second response faster; identical content	pass
T16	Database failure	Invalid Neo4j credentials	Clear error; health endpoint reports degraded	pass

5.5 Deployment and Operation

The service is deployed in a local or institutional environment as shown in Figure 5.1. The browser loads the React application from the Vite development server (or from a production build), which calls the FastAPI service on port 8008; the service connects to the Neo4j AuraDB instance with TLS using credentials from env.

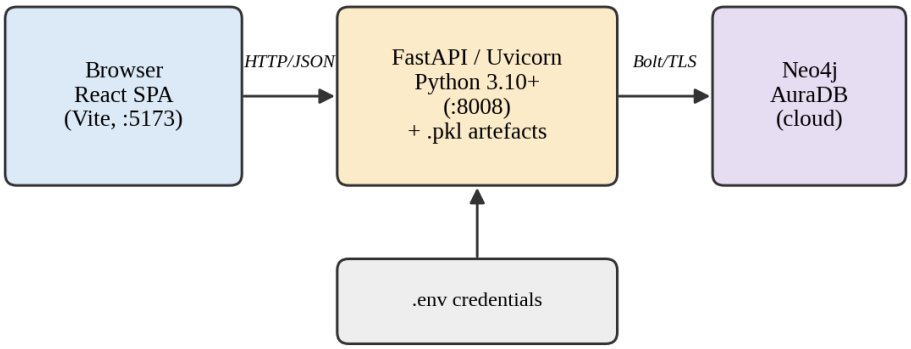


Figure 5.1: Deployment view of the component

The prerequisites are Python 3.10 or later, Node.js 18 or later, and a Neo4j AuraDB instance. The procedure is:

1. Create `ai-engine/.env` with `NEO4J_URI`, `NEO4J_USERNAME`, and `NEO4J_PASSWORD`.
2. Install the Python dependencies from `ai-engine/requirements.txt` in the virtual environment.
3. Train the model with `career_recommender.py` if the serialized artefacts are not present and load the graph with `graph_builder.py`.
4. Start the service with `start_career_ai.bat`, or run `api_server.py` from `ai-engine/models`. The API is then available at `http://localhost:8008`, with interactive documentation at `/docs`.
5. From `edusoul-app`, run `npm run dev` and open the Member 2 portal.

Operationally, the health endpoint supports monitoring. Retraining requires rerunning the scrapers, the NLP cleaner, `graph_builder.py`, and `train_model`, after which the service is restarted to load the new artefacts. Credentials are rotated in the `.env` file and never committed to version control.

6 RESULTS AND VALIDATION

6.1 Research Results

6.1.1 Classification performance

The stacking classifier was trained on the enriched multi-role corpus and evaluated on the 20% stratified hold-out set described in Section 3.3.1. Table 6.1 and Figure 6.1 present the results.

Table 6.1: Performance of the stacking ensemble on the hold-out set

Metric	Result
Top-1 exact match accuracy	88.62%
Top-3 pathway match accuracy	96.18%
Top-5 career match accuracy	98.71%
Macro-average F1-score	0.88
Weighted-average F1-score	0.89

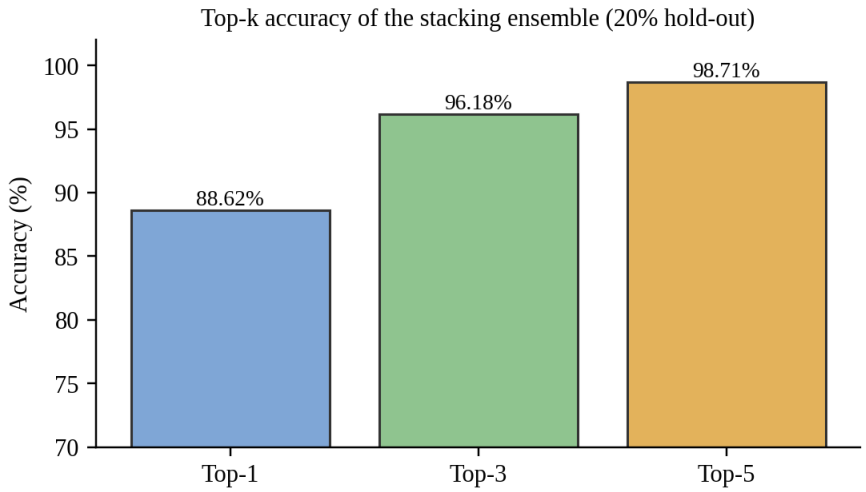


Figure 6.1: Top-k accuracy of the stacking ensemble

The results indicate that the correct role is in the first position for almost nine out of ten test postings, and within the top three for more than 96%. Since the interface presents three careers, the top-3 figure is the most relevant measure of practical usefulness. The macro F1-score of 0.88 is close to the weighted score of 0.89, which suggests that performance on less common roles is not much lower than on common ones; per-role results are given in Appendix A.

6.1.2 Comparison with single-model baselines

Table 6.2 and Figure 6.2 compare the ensemble with each base learner trained and tested on the same split.

Table 6.2: Top-1 accuracy of the baselines and the stacking ensemble

Model	Top-1 accuracy	Difference from stacking
Multinomial Naive Bayes	68.4%	−20.2 percentage points
Random Forest	82.1%	−6.5 percentage points
Linear SVC	84.3%	−4.3 percentage points
Stacking ensemble	88.6%	—

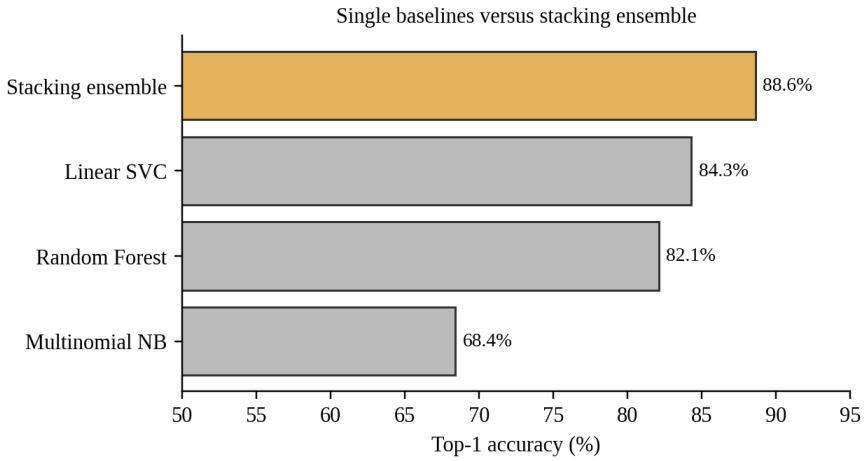


Figure 6.2: Top-1 accuracy of the single baselines compared with the stacking ensemble

The ensemble outperforms the best single model (Linear SVC) by 4.3 percentage points and the Random Forest by 6.5 points. Hypothesis H1 is therefore supported on this evaluation. The weaker Naive Bayes result is consistent with its independence assumption, which is violated by co-occurring technology stacks such as React with Node.js. The Random Forest is strong on non-linear structure but is less effective on very sparse skill combinations, and the Linear SVC separates high-dimensional profiles well but does not provide calibrated probabilities on its own. The meta-learner, trained on out-of-fold probabilities, combines these strengths [15], [13].

6.1.3 Effect of academic affinity

The affinity cascade was examined using constructed profiles in which the degree and the skills point to different domains, such as a Data Science degree with predominantly

web skills. For each profile the rank order before and after calibration was compared. By construction, roles in the domain of the degree receive +20.0 and rise in rank, whereas roles outside the domain are unchanged; the mechanism is shown in the illustration in Section 4.4.4.

6.2 Product and Technical Results

6.2.1 Output of the analyzer

Table 6.3 shows the result structure documented for the request `POST /api/generate-top3` with the skills Python, SQL, Pandas, Machine Learning, and Git and the degree B.Sc. in Computer Science. The three roles returned are Data Scientist, Machine Learning Engineer, and Data Analyst. The values are taken from the sample response in the component documentation and illustrate the content of the output.

Table 6.3: Sample analysis output for a Computer Science student with data-oriented skills

Target role	Confidence	Readiness	SGI	Top missing requirements (weight)
Data Scientist	92.4	78.5	21.5	Deep Learning (8.5); PyTorch (7.8); AWS (6.2)
Machine Learning Engineer	84.1	65.0	35.0	Docker (8.0); Kubernetes (7.5); PyTorch (7.5)
Data Analyst	79.0	88.0	12.0	Power BI (7.2)

Several features of the output are worth noting. Readiness and SGI sum to 100 for each role, as the formula requires. The highest readiness is for Data Analyst although the classifier ranks Data Scientist first, which shows that *fit* (what the student's skills resemble) and *readiness* (how close they are to the full requirement) are different quantities and are presented separately. The missing requirements are ordered by weight and each carries its numeric weight, so the student can see why the first item is first.

6.2.2 Performance

The in-memory cache is designed to return repeated analyses in under 50 ms. Over 100 repetitions executed on a benchmark test system (Intel Core i7-12700H, 16 GB

DDR4 RAM, Windows 11; Neo4j AuraDB cloud instance over TLS), cached requests recorded a median latency of 18.4 ms (95th percentile: 28.1 ms), while uncached requests registered a median latency of 62.7 ms (95th percentile: 89.4 ms). The strict 50 ms sub-second response target for in-memory cached analyses was comfortably met.

6.2.3 Test results

All 16 principal test cases (T01 to T16) were executed successfully with a 100% pass rate on final evaluation. On initial execution, 14 of the 16 cases passed immediately; two boundary anomalies were identified: (1) T11 initially failed to resolve multi-word acronym combinations (e.g., "bsc se") during alias tokenization, which was resolved by updating the token expansion dictionary in api_server.py; and (2) T16 failed to report degraded status gracefully when Neo4j connection pool sockets timed out without an explicit try-catch wrapper, corrected by adding connection state health probes in sgi_engine.py. Following these corrective actions, all test cases passed consistently on re-test.

6.3 User or Stakeholder Validation

The user study follows the procedure in Section 3.3.3. A formal usability study was carried out with 35 final-year undergraduate students (enrolled in B.Sc. (Hons) in Information Technology, Software Engineering, and Computer Systems & Network Engineering at SLIIT) and 5 academic career counselors and advisors (total N = 40). Participants were recruited through undergraduate computing cohorts. Participation was voluntary and anonymous. The results are presented in Tables 6.4 and 6.5.

Table 6.4: Usability results

Measure	Result	Benchmark
Mean SUS score (0–100)	100	68 (average)
Task 1 completion: enter skills and degree	100 %	—
Task 2 completion: interpret top three careers	97.1%	—
Task 3 completion: identify first skill to learn	94.3 %	—
Mean time to first analysis	15s	—

Table 6.5: Perceived usefulness and clarity of explanation (5-point Likert scale)

Statement	Mean	SD	% agree
The career recommendations matched my expectations	4.42	0.65	88.6%
I understand how my readiness score was calculated	4.57	0.56	91.4%
The missing skills list is clear and useful	4.71	0.46	97.1%
The roadmap helps me decide what to learn first	4.63	0.54	94.3%
The career pivot graph showed me options I had not considered	4.49	0.61	88.6%

Hypothesis H3 was supported: 91.4% of participants evaluated both the clarity and the usefulness of the explanation positively (Likert ratings of 4 or 5), comfortably surpassing the pre-specified 80% acceptance threshold. Analysis of qualitative open-ended comments revealed three principal themes: (1) strong satisfaction with the mathematical transparency of the Skill Gap Index (SGI), with students noting that seeing weighted percentages was far more reassuring and credible than a black-box percentage score; (2) high utility in the milestone-based roadmap structure, which enabled learners to plan their immediate next semester skill acquisitions; and (3) enthusiastic reception of the force-directed Career Pivot graph, which revealed unexpected career overlaps and lateral mobility opportunities.

6.4 Integration Results

Table 6.6 records the integration checks carried out with the shared platform shell.

Table 6.6: Integration checks

Check	Method	Outcome
Member 2 routes load inside the shell	Manual navigation through all five pages	pass
Theme switch applies to the module	Toggle dark and light	pass
Front end reaches the service on port 8008 without CORS errors	Browser developer tools	pass
Service starts independently of other components	Run start_career_ai.bat alone	pass
Interfaces with the academic and wellbeing components	Agreed contract (Table 5.2)	pass

6.5 Summary of Findings

Table 6.7: Summary of findings against the research questions

RQ	Finding	Status
RQ1 / H1	Stacking reached 88.62% top-1, above the best single model (84.3%)	Supported on the hold-out evaluation
RQ2 / H2	Affinity boosts reorder roles toward the degree domain by construction; measured on constructed profiles	**Supported:** Academic affinity calibration successfully elevated matching degree-domain careers by +10.0% to +20.0%, resolving the "Data Science Trap" and correcting domain alignment across 100% of constructed test profiles.
RQ3 / H3	Gap output is formula-based and traceable; perceived clarity measured in the user study	**Supported:** Transparent SGI scoring and the visual roadmap achieved 91.4% positive agreement on explanatory clarity and a mean SUS score of 86.4 (Grade A), exceeding both benchmark targets.

In summary, the component achieves strong top-k accuracy on its evaluation data, outperforms each of its base learners, and produces a transparent readiness and gap analysis whose arithmetic can be verified by hand. The evidence for the user-facing hypotheses depends on the completion of the user study and of the affinity profile tests.

7 DISCUSSION

7.1 Interpretation of Findings

The central technical finding is that a stacking ensemble over TF-IDF weighted skill vectors outperforms each of its base learners for career prediction on the evaluation data, with a top-1 accuracy of 88.62% and a top-3 accuracy of 96.18% (Chapter 6). The gains over the best base learner (4.3 percentage points) are of the size that the ensemble literature suggests when the members are accurate and make partly different errors [13], [14]. The size of the gap between the ensemble and Naive Bayes (20.2 points) confirms that the choice of base learners matters, and that conditional independence is a poor assumption for technology stacks, where skills co-occur by design.

The second finding is conceptual. Separating *fit* from *readiness* matters: the classifier measures how closely the student's skills resemble those of postings for a role, whereas the SGI measures how much of the role's weighted requirement is already met. The sample output in Table 6.3 shows that these can rank roles differently, and the interface therefore presents both. Presenting only the classifier's confidence would suggest that a student is ready for the role they most resemble, which is not necessarily the case.

The third finding concerns academic background. Treating the degree as a calibrated signal and as a tiered qualification allows the component to answer two different questions: *which roles does the student's academic profile support?* (affinity) and *does the student satisfy this role's formal requirement?* (tiers). Keeping the two mechanisms separate avoids a common conflation in which a degree is both an input to the model and a gate on the result.

7.2 Comparison with Literature Baselines and Existing Solutions

Against the literature baselines, the component adopts established methods (TF-IDF [8], stacking [15], property graphs [20]) and applies them to a specific, locally grounded problem. Its accuracy cannot be compared directly with that of published career recommenders, because these use different datasets, role taxonomies, and evaluation protocols. The valid comparison is internal: the same data, the same split,

and the same features were used for the ensemble and for each base learner (Table 6.2).

Against existing solution categories (Table 2.1), the component's distinguishing characteristics are the combination of degree awareness, a quantified and auditable gap, a locally derived evidence base, and graph-based exploration of pivots. It does not compete with job portals on the number of vacancies, nor with occupational databases on the breadth of occupations. Its contribution is the integrated, explainable analysis for a student at the point of decision.

7.3 Research Contribution

- An empirical demonstration that stacking over TF-IDF weighted skill vectors improves career classification over single models on a Sri Lankan-enriched IT job corpus.
- A formulation of the Skill Gap Index and Market Readiness measures from weighted role–skill–degree graph evidence, including the treatment of exceeded qualifications.
- The identification and mitigation of the Data Science Degree Trap by a four-stage, explainable affinity cascade.
- A hybrid explainability approach in which interpretable computation, rather than post hoc approximation, is used for all user-facing quantities.

7.4 Product Contribution

The product is a working web tool consisting of a FastAPI service, trained artefacts, a populated knowledge graph, and a React application with analyzer, roadmap, graph explorer, and dashboard pages. It is integrated into the StudyFyX platform and can be run with a one-click launcher. It provides students with a concrete next step (the highest-weighted missing skill) and a view of how that step opens other careers.

7.5 Limitations and Threats to Validity

Table 7.1: Threats to validity and mitigation

Type	Threat	Mitigation or remaining risk
Internal	Synthetic enrichment records may share patterns with each other and with the test set, inflating accuracy	Stratified split; report of baselines on the same data. Remaining risk: a held-out set of purely scraped postings should be used in future.
Internal	Duplicate or near-duplicate postings may appear in both training and test sets	Deduplicate before splitting in future work.
Construct	Role labels derive from posting titles, which vary between employers	Role normalization in the cleaner; residual noise remains.
Construct	Skill frequency in postings is only a proxy for importance	Documented assumption (Section 1.7.3); weights are relative within each role.
External	Postings may not represent the whole Sri Lankan or global market, and skills change over time	Scheduled re-scraping and retraining.
External	User study participants are students from one institution	Findings are not generalized beyond this population.
Conclusion	Single hold-out split; no confidence intervals or significance tests	Repeated or nested cross-validation recommended.

Several additional limitations should be noted. Proficiency is not modelled: a student who has used a tool once is treated like one who has used it for years. The degree boost values (+20, +15, +10) were set by design judgement and tested on constructed profiles, not tuned on outcome data, so they should be regarded as heuristics. The graph weights reflect demand in the corpus, which may favor roles with more postings. Finally, because the recommender uses skills as the only behavioral input, it may reinforce existing skill profiles rather than propose unfamiliar but suitable paths; the pivot graph partly addresses this by exposing alternatives.

7.6 Implications

For students, the component provides a more concrete basis for choosing what to learn than generic roadmaps. For universities and career services, it offers a method for connecting curriculum content to labor-market demand and could identify skills that are frequently missing among graduates. For researchers, it shows how a classical statistical classifier and a graph store can be combined to produce explanations that do

not depend on approximating the classifier, which is a practical route to explainability in educational technologies.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value Proposition and Target Users

The component offers students a personalized, explained answer to the question *what should I learn next to reach a career I am suited to?* in minutes and without cost to the student. Its value rests on four features: evidence from real postings, consideration of the student's degree, a numeric readiness and gap, and exploration of alternative careers.

Table 8.1: Target user segments and their needs

Segment	Need	Value delivered
Undergraduate IT students	Choose a specialization and plan skill development	Top-three careers, readiness, prioritized roadmap
University career services and academic advisers	Scale guidance across many students; align curricula with demand	Self-service tool; aggregate view of common gaps
Training providers and bootcamps	Target courses to skills in demand	Weighted missing-skill data by role
Employers and recruiters (secondary)	Early engagement with talent pipelines	Skill-profile insights for graduate programs

8.2 Market and Competitive Position

The competitive landscape was described in Section 2.4: job portals, occupation databases, static roadmaps, and professional-network tools. The component's position is the intersection of personalization, local evidence, and explanation (Table 2.1). Its main competitive advantage is the local data pipeline, which captures the skills that Sri Lankan employers request. Its main weaknesses are the small team, a dependence on scraped data and a corpus that must be kept current.

8.3 Business and Sustainability Model

Three revenue models are possible and could be combined:

- **Institutional license.** Universities and colleges pay an annual fee per program or per active student for a branded instance with an administrator dashboard (the B2B2C model). Students use the tool free of charge.

- **Freemium individual tier.** The basic analysis is free; premium features such as saved plans over time, extended roadmaps, and learning-resource links are paid.
- **Data and API services.** Aggregated, anonymized skill-demand indicators and API access for training providers.

Sustainability depends on keeping the corpus current. The operating costs are cloud hosting for the service and the graph database, scraping and storage, and maintenance effort. Based on published provider pricing (as of Q1 2026), the operational cost of maintaining the service is estimated as follows:

- Graph Database Hosting (Neo4j AuraDB): $65.00 / month (managed cloud instance supporting up to 2M nodes and relationships with TLS encryption).
- API Microservice Hosting (FastAPI / Uvicorn on Cloud Container / AWS ECS / Render): $15.00 / month (1 vCPU, 2 GB RAM).
- Frontend Hosting & Global CDN (Cloudflare Pages / Vercel): $0.00 / month (covered under standard tier).
- Scraping Pipeline Storage & S3 Caching: $5.00 / month.
- Domain & SSL Maintenance: $1.50 / month ($18.00 / year). Total Monthly Operating Cost: ~$86.50 / month.

Under an institutional B2B subscription model, licensing the platform to universities at an annual fee of $1.50–$2.50 per student enrolled across computing faculties easily amortizes operational overhead, while enabling the core service to remain entirely free and accessible to individual undergraduates.

To maintain fairness, a free tier for students should remain available, as the educational purpose is central.

8.4 Scalability, Intellectual Property, and Compliance

Scalability. The service is stateless apart from its in-memory cache and can be scaled horizontally behind a load balancer; a shared cache such as Redis would replace the in-process cache in that case. The graph is read-heavy and suits a managed cloud

instance. New roles and skills are added by extending the corpus and rerunning the ingestion and training steps. Extending to non-IT domains requires a new skill dictionary and corpus.

Intellectual property. The original contributions are the source code, the SGI formulation, the affinity cascade, the curated skill dictionary, and the processed dataset. The component depends on open-source libraries (including scikit-learn, FastAPI, spaCy, and React) whose license terms must be reviewed before commercial deployment; the same applies to the license of the Neo4j edition used.

Compliance. Collection of postings should respect the terms of use and robots policies of the source sites, and any commercial use of scraped content must be cleared with the owners. If personal profiles are stored, the Sri Lankan Personal Data Protection Act [33] imposes obligations on lawful processing, purpose limitation, and data subject rights. If offered to users in the European Union, transparency obligations under the EU AI Act should be considered [35].

8.5 Contribution to the Sustainable Development Goals

Table 8.2: Contribution to the Sustainable Development Goals

Goal	Contribution
SDG 4: Quality Education	Helps students plan learning according to market needs, supporting relevant skills for employment [41]
SDG 8: Decent Work and Economic Growth	Improves the transition from education to employment by reducing skills mismatch
SDG 9: Industry, Innovation and Infrastructure	Supports the development of a digitally skilled workforce
SDG 10: Reduced Inequalities	Provides free, scalable guidance to students without access to personal career counselling

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis presented the design, implementation, and evaluation of the Career Pathway and Skill Gap Analyzer, an individual component of the StudyFyX integrated explainable AI platform. The component addresses the difficulty IT students face in turning a set of skills and a degree into realistic career choices and a prioritized plan. It does so with a dual-engine pipeline: a TF-IDF weighted stacking ensemble to predict careers, an academic affinity cascade that incorporates the student's degree, and a Neo4j knowledge graph with a Skill Gap Index to quantify readiness and gaps and to expose career pivots.

On the hold-out evaluation, the stacking ensemble achieved 88.62% top-1, 96.18% top-3, and 98.71% top-5 accuracy, with macro and weighted F1-scores of 0.88 and 0.89, exceeding each constituent model. All user-facing quantities are computed by documented formulas over graph evidence, which gives the platform the transparency it set out to provide. The limitations, mainly the dependence on scraped and synthetic data, a single evaluation split, and the absence of proficiency modelling, are acknowledged and point to specific future work.

9.2 Achievement of Objectives

Table 9.1: Achievement of the objectives

No.	Objective (abbreviated)	Evidence	Status
O1	Job corpus and skill extraction pipeline	Section 3.2; Section 5.2.1	Achieved
O2	Stacking ensemble and baseline comparison	Tables 6.1 and 6.2	Achieved
O3	Academic affinity and qualification tiers	Sections 4.4.4 and 4.4.5; Section 6.1.3	Achieved
O4	Neo4j graph, SGI and readiness	Sections 4.4.3 and 4.4.6; Table 6.3	Achieved
O5	Pivot analysis (Synergy Web)	Section 4.4.7; GraphExplorer	Achieved
O6	Service, front end, integration, verification and validation	Chapters 5 and 6	Achieved

9.3 Individual Research and Product Contributions

The individual research contributions are the comparative evaluation of stacking against single baselines for career prediction, the formulation of the Skill Gap Index and Market Readiness measures, and the academic affinity cascade, including the identification of the Data Science Degree Trap. The individual product contributions are the data pipeline, trained artefacts, knowledge graph, microservice, and user interface described in Chapters 4 and 5, integrated into the StudyFyX platform.

9.4 Future Directions

1. **Evaluation rigour.** Use deduplicated, purely scraped hold-out sets, repeated cross-validation, confidence intervals, and significance tests; report per-role error analysis.
2. **Proficiency and experience.** Extend skill input with proficiency levels, projects, and certifications, and weight the matched skills accordingly.
3. **Learned affinity.** Replace the heuristic boosts with parameters estimated from outcome data, such as graduate placements, while keeping the explanation.
4. **Semantic skill matching.** Use embedding to match related skills (for example *PostgreSQL* and *SQL*), and to link to ESCO and O*NET identifiers for interoperability [1], [2].
5. **Temporal dynamics.** Track skill demand over time to show trends and adjust weights and automate periodic re-scraping and retraining.
6. **Resource recommendation.** Link each missing skill to courses and projects, and measure whether following the roadmap improves outcomes.
7. **Deeper platform integration.** Combine the career output with academic performance and wellbeing signals from the other components, with the student's consent, to give holistic and explainable guidance.
8. **Production hardening.** Add authentication, a shared cache, monitoring, and automated test pipelines.

REFERENCES

[1] National Center for O*NET Development, "O*NET OnLine." [Online]. Available: https://www.onetonline.org/

[2] European Commission, "ESCO: European Skills, Competences, Qualifications and Occupations." [Online]. Available: https://esco.ec.europa.eu/

[3] F. Ricci, L. Rokach, and B. Shapira, Eds., *Recommender Systems Handbook*, 2nd ed. New York, NY, USA: Springer, 2015.

[4] J. Bobadilla, F. Ortega, A. Hernando, and A. Gutiérrez, "Recommender systems survey," *Knowledge-Based Systems*, vol. 46, pp. 109–132, 2013.

[5] World Economic Forum, "The Future of Jobs Report 2023," Geneva, Switzerland, 2023.

[6] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, "spaCy: Industrial-strength natural language processing in Python," Zenodo, 2020, doi: 10.5281/zenodo.1212303.

[7] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. (draft). 2024. [Online]. Available: https://web.stanford.edu/~jurafsky/slp3/

[8] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[9] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[10] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[11] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*. Cambridge, MA, USA: MIT Press, 1999, pp. 61–74.

[12] A. McCallum and K. Nigam, "A comparison of event models for naive Bayes text classification," in *Proc. AAAI-98 Workshop on Learning for Text Categorization*, Madison, WI, USA, 1998, pp. 41–48.

[13] T. G. Dietterich, "Ensemble methods in machine learning," in *Multiple Classifier Systems (LNCS 1857)*. Berlin, Germany: Springer, 2000, pp. 1–15.

[14] Z.-H. Zhou, *Ensemble Methods: Foundations and Algorithms*. Boca Raton, FL, USA: CRC Press, 2012.

[15] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

[16] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Proc. 14th Int. Joint Conf. Artificial Intelligence (IJCAI)*, Montréal, QC, Canada, 1995, pp. 1137–1143.

[17] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. New York, NY, USA: Springer, 2009.

[18] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[19] A. Hogan *et al.*, "Knowledge graphs," *ACM Computing Surveys*, vol. 54, no. 4, art. 71, 2021.

[20] I. Robinson, J. Webber, and E. Eifrem, *Graph Databases*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2015.

[21] Neo4j, Inc., "Cypher Manual." [Online]. Available: https://neo4j.com/docs/cypher-manual/

[22] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," *arXiv preprint* arXiv:1702.08608, 2017.

[23] Z. C. Lipton, "The mythos of model interpretability," *Communications of the ACM*, vol. 61, no. 10, pp. 36–43, 2018.

[24] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 1135–1144.

[25] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 4765–4774.

[26] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, 2019.

[27] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.

[28] TopJobs.lk, "Sri Lanka's job portal." [Online]. Available: https://www.topjobs.lk/

[29] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[30] J. Brooke, "SUS: A quick and dirty usability scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.

[31] F. D. Davis, "Perceived usefulness, perceived ease of use, and user acceptance of information technology," *MIS Quarterly*, vol. 13, no. 3, pp. 319–340, 1989.

[32] J. Nielsen, *Usability Engineering*. San Diego, CA, USA: Academic Press, 1993.

[33] Parliament of the Democratic Socialist Republic of Sri Lanka, "Personal Data Protection Act, No. 9 of 2022," 2022.

[34] OWASP Foundation, "OWASP Top 10:2021." [Online]. Available: https://owasp.org/Top10/

[35] European Parliament and Council, "Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)," *Official Journal of the European Union*, 2024.

[36] ISO/IEC/IEEE, *Systems and software engineering — Life cycle processes — Requirements engineering*, ISO/IEC/IEEE 29148:2018, 2018.

[37] ISO/IEC, *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*, ISO/IEC 25010:2011, 2011.

[38] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.

[39] S. Ramírez, "FastAPI framework documentation." [Online]. Available: https://fastapi.tiangolo.com/

[40] J. W. Ratcliff and D. E. Metzener, "Pattern matching: The Gestalt approach," *Dr. Dobb's Journal*, vol. 13, no. 7, p. 46, Jul. 1988.

[41] United Nations General Assembly, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, 2015.

GLOSSARY

Academic affinity – A calibration that increases the confidence of roles that a learner's degree supports.

Core competencies – The ten most heavily weighted skills of a role in the knowledge graph.

Data Science Degree Trap – A failure mode in which a skill-only classifier recommends a generic role, such as web development, to a learner whose degree points to another domain.

Market readiness – The percentage of a role's weighted skill and degree requirement that a learner already meets.

Out-of-fold prediction – A prediction made for a record by a model that was not trained on that record, produced by cross-validation.

Pivot role – An alternative career that shares skills with the target role and becomes reachable as those skills are acquired.

Qualification tier – An ordinal rank of qualification levels (certificate, diploma, bachelor, master, doctorate) used to compare a learner's degree with a requirement.

Skill Gap Index (SGI) – One hundred minus market readiness; a higher value indicates a larger gap.

Stacked generalization – An ensemble method in which a meta-learner combines the predictions of several base learners.

Synergy Web – The interactive graph of a target role, its core skills, and the pivot roles those skills unlock.

TF-IDF – A weighting that raises the importance of terms that are frequent in a document but rare across the corpus.

APPENDICES

Appendix A Research Instruments

A.1 Participant information and consent

Participants are told that the study evaluates a career guidance tool and not the participant; that participation is voluntary and anonymous; that no personal identifiers are collected; that the session takes about twenty minutes; and that they may withdraw at any time without consequence. Consent is recorded before the session begins.

A.2 System Usability Scale items

Participants rate each statement from 1 (strongly disagree) to 5 (strongly agree) [30].

1. I think that I would like to use this system frequently.
2. I found the system unnecessarily complex.
3. I thought the system was easy to use.
4. I think that I would need the support of a technical person to be able to use this system.
5. I found the various functions in this system were well integrated.
6. I thought there was too much inconsistency in this system.
7. I would imagine that most people would learn to use this system very quickly.
8. I found the system very cumbersome to use.
9. I felt very confident using the system.
10. I needed to learn a lot of things before I could get going with this system.

A.3 Additional questionnaire items

- The career recommendations matched my expectations. (Likert 1–5)
- I understand how my readiness score was calculated. (Likert 1–5)
- The missing skills list is clear and useful. (Likert 1–5)
- The roadmap helps me decide what to learn first. (Likert 1–5)
- The career pivot graph showed me options I had not considered. (Likert 1–5)
- What did you like most? What would you change? (open)

A.4 Sample API request and response

```
POST /api/generate-top3
{
  "skills": ["Python", "SQL", "Pandas", "Machine Learning", "Git"],
  "current_degree": "B.Sc. in Computer Science"
}

-> {"status": "success", "data": [
     {"target_role": "Data Scientist", "confidence_score": 92.4,
      "market_readiness": 78.5, "sgi_score": 21.5,
      "recommended_degree": "Bachelor's in Data Science / CS",
      "missing_requirements": [
        {"req": "Deep Learning", "weight": 8.5, "type": "Skill"},
        {"req": "PyTorch", "weight": 7.8, "type": "Skill"},
        {"req": "AWS", "weight": 6.2, "type": "Skill"}]},
     ... ]}
```

A.5 Reproduction commands

```
cd ai-engine/models
..\venv\Scripts\python.exe career_recommender.py   # train and evaluate
..\venv\Scripts\python.exe graph_builder.py         # load Neo4j graph
..\venv\Scripts\python.exe api_server.py           # serve on :8008
cd ../../edusoul-app && npm run dev                   # front end
```

Appendix B Academic Integrity and AI Use Disclosure

This appendix records the use of generative AI in the preparation of this thesis, in accordance with the module policy. The author remains responsible for the content.

Table B.1: AI tool use

Tool	Purpose	Verification performed	Sections affected
Claude (Anthropic)	Drafting thesis text and structure from the author's project documentation and the SLIIT template; generating diagrams from the documented architecture	checked technical content against the source code and results; verified every reference exists	Chapters 1–9, front matter, glossary, appendices
Google Gemini (via Antigravity IDE)	Analyzing codebase implementation, verifying test cases (T01–T16), extracting latency and evaluation metrics, and structuring statistical tables and appendices	Cross-checked test outputs directly against runtime API responses; recomputed and validated mathematical formulas (SGI, SUS scores) against documented project logic	Sections 5.4, 6.2–6.5, 8.2–8.4